```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor

from sklearn.metrics import mean_absolute_error
from sklearn.metrics import mean_squared_error
from sklearn.metrics import r2_score

# Updated to use an available dataset
df = pd.read_csv("/content/sample_data/california_housing_train.csv")

df.head()

print("Shape:", df.shape)

df.info()

df.describe()

df.isnull().sum()

# Updated target variable to 'median_house_value'
X = df.drop("median_house_value", axis=1)
y = df["median_house_value"]

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)

model = LinearRegression()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)

rmse = np.sqrt(mean_squared_error(y_test, y_pred))

r2 = r2_score(y_test, y_pred)

print("Baseline Model Performance")
print("--------------------------")
print("MAE :", mae)
print("RMSE:", rmse)
print("R²  :", r2)

train_pred = model.predict(X_train)
test_pred = model.predict(X_test)

train_r2 = r2_score(y_train, train_pred)
test_r2 = r2_score(y_test, test_pred)

print("Training R²:", train_r2)
print("Testing R² :", test_r2)

dt_model = DecisionTreeRegressor(random_state=42)

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

dt_rmse = np.sqrt(mean_squared_error(y_test, dt_pred))
dt_r2 = r2_score(y_test, dt_pred)

print("Decision Tree")
print("RMSE:", dt_rmse)
```

```
print("R²:", dt_r2)

dt_model = DecisionTreeRegressor(
    max_depth=5,
    random_state=42
)

dt_model.fit(X_train, y_train)

dt_pred = dt_model.predict(X_test)

print("RMSE:", np.sqrt(mean_squared_error(y_test, dt_pred)))
print("R²:", r2_score(y_test, dt_pred))

scores = cross_val_score(
    dt_model,
    X,
    y,
    cv=5,
    scoring="r2"
)

print("Cross-validation scores:", scores)
print("Mean R²:", scores.mean())

param_grid = {
    "max_depth": [3, 5, 7, 10, None],
    "min_samples_split": [2, 5, 10],
    "min_samples_leaf": [1, 2, 4]
}

grid_search = GridSearchCV(
    estimator=DecisionTreeRegressor(random_state=42),
    param_grid=param_grid,
    cv=5,
    scoring="r2",
    n_jobs=-1
)

grid_search.fit(X_train, y_train) # Fit the GridSearchCV object

print("Best Parameters:")
print(grid_search.best_params_)

print("Best Cross-Validation R²:")
print(grid_search.best_score_)

best_model = grid_search.best_estimator_

tuned_pred = best_model.predict(X_test)

tuned_mae = mean_absolute_error(y_test, tuned_pred)

tuned_rmse = np.sqrt(
    mean_squared_error(y_test, tuned_pred)
)

tuned_r2 = r2_score(y_test, tuned_pred)

print("Tuned Decision Tree Performance")
print("--------------------------------")
print("MAE :", tuned_mae)
print("RMSE:", tuned_rmse)
print("R²  :", tuned_r2)

results = pd.DataFrame({
    "Model": [
        "Linear Regression",
        "Decision Tree",
        "Tuned Decision Tree"
    ],
    "RMSE": [
        rmse,
        dt_rmse,
        tuned_rmse
    ],
    "R2": [
        r2,
        dt_r2,
        tuned_r2
    ]
})
```

```
results

plt.figure(figsize=(8, 5))

plt.scatter(y_test, tuned_pred)

plt.xlabel("Actual Price")
plt.ylabel("Predicted Price")
plt.title("Actual vs Predicted Values")

plt.show()
```

```
Shape: (17000, 9)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 17000 entries, 0 to 16999
Data columns (total 9 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   longitude           17000 non-null  float64
 1   latitude            17000 non-null  float64
 2   housing_median_age  17000 non-null  float64
 3   total_rooms         17000 non-null  float64
 4   total_bedrooms      17000 non-null  float64
 5   population          17000 non-null  float64
 6   households          17000 non-null  float64
 7   median_income       17000 non-null  float64
 8   median_house_value  17000 non-null  float64
dtypes: float64(9)
memory usage: 1.2 MB
Training data: (13600, 8)
Testing data: (3400, 8)
Baseline Model Performance
--------------------------
MAE : 49983.47465122931
RMSE: 68078.32552452553
R²  : 0.6636396350243869
Training R²: 0.6352694644505287
Testing R² : 0.6636396350243869
Decision Tree
RMSE: 69597.05661237321
R²: 0.6484647902191261
RMSE: 75278.87814901918
R²: 0.5887240052926918
Cross-validation scores: [0.31355786 0.3810472  0.42411402 0.32064645 0.54191897]
Mean R²: 0.3962568994896506
Best Parameters:
{'max_depth': 10, 'min_samples_leaf': 4, 'min_samples_split': 10}
Best Cross-Validation R²:
0.7113691227175145
Tuned Decision Tree Performance
-------------------------------
MAE : 41073.516770738126
RMSE: 61026.343533680345
R²  : 0.7297151000095745
```